CSC3110 Final Exam — Comprehensive Study Guide

> **How to use this note.** The exam is built from nine recurring question types (straight from the review deck). Each section below follows the same rhythm: **the idea → an analogy → the algorithm → a fully worked example → the traps that lose points.** Read a section, then cover the worked example and redo it yourself on paper. If you can reproduce every worked example here from a blank page, you're ready. All numbers below are machine-verified.

---

0. The Big Picture — which technique, and why

Every algorithm on this exam is one answer to a single question: *"How do I relate this problem to a smaller or different version of itself?"* Keeping the map in your technique map in your head is worth more than any single algorithm, because the exam rewards recognizing **which tool fits**.

Technique	Relationship to a smaller problem	Signature algorithms (this exam)
Decrease & Conquer	Solve **one** smaller instance, then extend	Topological sort (source removal), insertion sort
Divide & Conquer	Split into **several** subproblems, solve, combine	Quicksort / Hoare's partition, mergesort
Transform & Conquer	**Transform** into a better representation, then solve	Heap construction (heapify), presorting
Space–Time Trade-off	**Precompute & store** to make later work cheap	Comparison counting sort
Dynamic Programming	**Overlapping** subproblems solved once into a table	Coin-row, coin-change
Greedy	Sequence of locally-best, **irrevocable** choices	Prim's, Kruskal's MST
Backtracking / Branch-&-Bound	Build a **state-space tree**, prune with bounds	Assignment problem

The two traps the exam loves. (1) *DP vs Divide-and-Conquer* — both split a problem, but D&C subproblems are **independent**, whereas DP exists precisely **because** subproblems **overlap** (naive recursion would be exponential). (2) *Greedy vs DP* — greedy commits to one choice and never looks back (fast, but only correct when the problem has the right structure); DP considers **all** choices via a table and is always correct where it applies.

Complexity reference (know these cold)

Algorithm	Best	Average	Worst	Space
Topological sort (source removal / DFS)	Θ(V+E)	Θ(V+E)	Θ(V+E)	Θ(V)
Quicksort (Hoare partition)	Θ(n log n)	Θ(n log n)	Θ(n^2)	Θ(log n)
Bottom-up heap construction (heapify)	Θ(n)	Θ(n)	Θ(n)	Θ(1)
Heapsort	Θ(n log n)	Θ(n log n)	Θ(n log n)	Θ(1)
Comparison counting sort	Θ(n^2)	Θ(n^2)	Θ(n^2)	Θ(n)
Distribution counting sort	Θ(n+range)	Θ(n+range)	Θ(n+range)	Θ(n)
Prim's MST (heap + adj list)	Θ(E log V)	Θ(E log V)	Θ(E log V)	Θ(V)
Kruskal's MST	Θ(E log E)	Θ(E log E)	Θ(E log E)	Θ(V)
Coin-row / coin-change DP	Θ(n) / Θ(n·amount)	—	—	Θ(n)
Assignment (branch-and-bound)	exponential worst case; prunes in practice	—	—	Θ(state tree)

> One fact that shows up as a trick question: heapsort is Θ(n log n), but **building** the heap bottom-up is only **Θ(n)** — the cheap part.

---

1. Topological Sorting — Source-Removal Algorithm

The idea. Given a **directed acyclic graph (DAG)**, a topological sort is a linear ordering of vertices such that every edge points "forward" — if there's an edge u → v, then u comes before v in the list. It answers *"in what order can I do these tasks if some must happen before others?"* A topological order exists **iff** the graph is a DAG — a cycle makes ordering impossible. (See Cyclic vs Acyclic Graph; a directed graph with no cycles = DAG.)

Analogy. Getting dressed. Socks before shoes, shirt before jacket. Some items have no order between them (socks vs. shirt), but no valid order ever puts shoes before socks. A topological sort is any complete dressing sequence that respects every "must-come-before" rule.

Source-removal algorithm. 1. Find a **source** — a vertex with **in-degree 0** (nothing points to it). 2. Add it to the output list and **remove it** from the graph (delete it and all its outgoing edges). 3. Removing it drops the in-degree of its neighbors — new sources may appear. 4. Repeat until the graph is empty. If you ever get stuck with vertices left but no source, the graph has a **cycle** → no topological order.

Tie-breaking: when several sources exist at once, the convention is usually **alphabetical/lowest-label first** — follow whatever your instructor specified, and be consistent.

Worked example. Graph: A→B, A→C, A→E, B→D, C→B, D→E, E→C? — no, that would be a cycle. Use the acyclic version A→B, A→C, A→D, B→E, C→E, D→E:

Step	In-degree 0 (source)	Remove	Output so far
1	A	A → drops B,C,D to 0	A
2	B (lowest label)	B → drops E's count	A, B
3	C	C	A, B, C
4	D	D → E now in-degree 0	A, B, C, D
5	E	E	A, B, C, D, E

Topological order: A, B, C, D, E. (Different valid orders exist — e.g. A, D, C, B, E — because B, C, D are mutually unordered. Any order respecting all edges is correct.)

Traps. (1) Picking a vertex that still has incoming edges — always re-check in-degrees after each removal. (2) Forgetting that a leftover graph with no source means a **cycle**, not a mistake in your arithmetic. (3) The alternative **DFS method** (push vertices onto a stack as they finish; the reversed pop order is the topological sort) gives the *same* Θ(V+E) — know both exist; your three-color DFS note is the cycle-detection cousin of this.

---

2. Array Partition — Hoare's Partition Algorithm

The idea. Partitioning is the engine inside **quicksort** (Divide & Conquer). Pick a **pivot**, then rearrange the array so everything ≤ pivot sits to its left and everything ≥ pivot sits to its right. After one partition the pivot is in its **final sorted position** — the "split point" — even though nothing else is sorted yet.

Analogy. Sorting a room of people by height around one "reference" person. You send shorter people left, taller people right. After the shuffle the reference person is standing exactly where they'll finish; the two groups on either side are still internally jumbled, to be sorted later.

Hoare's partition (Levitin version). Pivot p = A[l] (first element). Set i = l, j = r+1. - Scan **i rightward**, stopping at the first element **≥ p**. - Scan **j leftward**, stopping at the first element **≤ p**. - If i < j, **swap A[i] and A[j]**, continue. - When **i ≥ j**, stop and swap the pivot into place: **swap A[l] and A[j]**. Now j is the split point.

Worked example — one pass on `[42, 9, 26, 41, 56, 43, 33]`. Pivot p = 42, i starts left of index 1, j starts right of index 6.

Action	i	A[i]	j	A[j]	Array
i stops at first ≥42	4	56	—	—	42 9 26 41 **56** 43 33
j stops at first ≤42	—	—	6	33	42 9 26 41 56 43 **33**
i<j → swap A[4],A[6]					42 9 26 41 **33** 43 **56**
i advances to ≥42	5	43			
j retreats to ≤42			4	33	
i ≥ j → stop; swap A[l],A[j=4]					**33** 9 26 41 **42** 43 56

Result after one partition: `[33, 9, 26, 41, 42, 43, 56]`. Pivot **42 lands at index 4** (its final sorted spot); everything left is ≤ 42, everything right is ≥ 42.

Traps. (1) The **final** swap is with **A[j]**, not A[i] — mixing them up is the #1 error. (2) The comparisons are **≥** for i and **≤** for j; equal elements must stop the scan or you can run off the array. (3) After you find the split point, quicksort recurses on A[l..j-1] and A[j+1..r] — the pivot itself is done. (4) Worst case Θ(n²) happens on already-sorted input (pivot is always the extreme), average Θ(n log n).

---

3. Heap Construction — Bottom-Up Heapify

The idea. A **max-heap** is a complete binary tree (stored as an array) where every parent ≥ its children. Bottom-up construction turns an arbitrary array into a heap **in Θ(n) time** — this is *Transform & Conquer*: we transform the data into a better representation (a heap) so the next operation (extract-max) is cheap. Array indexing (0-based): node `i` has children `2i+1` and `2i+2`, parent `⌊(i−1)/2⌋`.

Analogy. A corporate reorg from the bottom up. You start at the lowest managers who actually have reports (the last non-leaf node) and, for each, if a subordinate outranks the boss, promote the strongest subordinate. You "sift" the weak manager down until they sit above only weaker people. Working upward, by the time you fix the CEO the whole hierarchy obeys "boss ≥ reports."

Algorithm. For `i` from the **last parent** (index ⌊n/2⌋ − 1) **down to 0**: sift A[i] down — repeatedly compare it with its **larger child**; if the child is bigger, swap, and continue from the child's position. Stop when the node is ≥ both children or becomes a leaf. Leaves (the second half of the array) are skipped — they're already trivial heaps.

Worked example — build a max-heap from `[31, 10, 24, 1, 12]` (n = 5, last parent = index 1).

Tree as given:

```
      31(0)
     /    \
  10(1)    24(2)
  /   \
1(3)  12(4)
```

- **i = 1 (value 10):** children are 1 (idx 3) and 12 (idx 4). Larger child = 12 > 10 → **swap 10 and 12**. Array → `[31, 12, 24, 1, 10]`. Node 10 is now a leaf; done.
- **i = 0 (value 31):** children are 12 (idx 1) and 24 (idx 2). Larger child = 24, but 31 ≥ 24 → **no swap**.

Final heap: `[31, 12, 24, 1, 10]` — one swap total.

```
      31
     /   \
   12    24
  /  \
 1    10
```

Traps. (1) Always compare against the **larger** of the two children, not the left child by default. (2) After a swap you must **keep sifting** from the new position (a single node can cascade several levels down) — the exam's larger array `[1,3,5,4,6,13,10,9,8,15,17]` heapifies to `[17,15,13,9,6,5,10,4,8,3,1]` through eight swaps precisely because nodes cascade. (3) "A step is a swap" — draw the tree/array **after each swap**, that's what earns partial credit. (4) Don't confuse building the heap (Θ(n)) with heapsort's repeated extract-max (Θ(n log n)).

---

4. Minimum Spanning Tree — Kruskal's Algorithm

The idea. An **MST** of a weighted connected graph is an **acyclic**, connected subgraph touching every vertex (a spanning **tree**, so exactly |V|−1 edges) with **minimum total edge weight** (see your Kruskal note). Kruskal's is **greedy by edge**: repeatedly grab the cheapest edge that doesn't create a cycle.

Analogy. Laying the cheapest possible road network to connect every town. You lay roads cheapest-first, but skip any road whose two towns are **already connected** by roads you've built — that road would just add a redundant loop. You stop once every town is on one network.

Algorithm. 1. Sort all edges by increasing weight. 2. Go through them in order. Add an edge **iff** it does **not** create a cycle (its two endpoints are in different components — track with union-find). 3. Stop when you have |V| − 1 edges. That set is the MST.

Why it's correct (the cut/cycle argument): if adding an edge closes a cycle, the heaviest edge in that cycle is never needed, so a minimum-weight connected subgraph is always acyclic.

Worked example. Vertices {A,B,C,D,E}. Sorted edges: **A–D(1), B–D(1), B–C(1), B–E(2), A–B(3), D–E(3), C–E(4), A–C(4), A–E(7)**. Need |V|−1 = 4 edges.

Edge	Weight	Cycle?	Action	MST edges	Total
A–D	1	no	**add**	{AD}	1
B–D	1	no	**add**	{AD, BD}	2
B–C	1	no	**add**	{AD, BD, BC}	3
B–E	2	no	**add**	{AD, BD, BC, BE}	5
A–B	3	yes (A,B already joined via D)	skip	—	—

Four edges reached → **stop**. **MST = {A–D, B–D, B–C, B–E}, total weight = 5.**

Traps. (1) You **skip** an edge that forms a cycle — you don't stop the algorithm. (2) Stop as soon as you have |V|−1 edges; don't keep going. (3) Sort **all** edges first; ties can be broken arbitrarily but be consistent. (4) The complexity Θ(E log E) is dominated by the **sort**, not the union-find.

---

5. Minimum Spanning Tree — Prim's Algorithm

The idea. Same MST goal, different greedy rule. Prim's is **greedy by vertex**: grow **one** tree outward from a start vertex, each step adding the **cheapest edge that connects a new vertex** to the tree already built (see your Prim note).

Analogy. A spreading vine from one seed. At every moment the vine reaches out along its single cheapest available tendril to grab **one** new, not-yet-reached point. It's always one connected plant — never separate pieces (that's the key difference from Kruskal's, which can grow several forest fragments that merge later).

Algorithm. 1. Start from any vertex; put it in the tree V_T. 2. Look at all edges crossing from V_T to outside vertices. Pick the **minimum-weight** such edge. 3. Add that edge and its new vertex to the tree. 4. Repeat until all vertices are in the tree (|V|−1 edges added).

Worked example. Same graph, start at **A**. Fringe = cheapest edge from the tree to each outside vertex.

Step	Tree V_T	Cheapest crossing edge	Add	Total
1	{A}	A–D(1)	D	1
2	{A,D}	B–D(1)	B	2
3	{A,D,B}	B–C(1)	C	3
4	{A,D,B,C}	B–E(2)	E	5

All 5 vertices in → **MST = {A–D, B–D, B–C, B–E}, weight 5** — same tree as Kruskal's here (MSTs coincide when edge weights make the choice unique).

Prim vs Kruskal — the exam distinction. Prim's keeps **one growing connected tree** and picks the cheapest edge **on its frontier**; Kruskal's picks the globally cheapest **remaining** edge and may build **several fragments** that later merge. Both are greedy, both give a valid MST. Prim's shines on **dense** graphs (Θ(E log V) with a heap), Kruskal's on **sparse** graphs (cost dominated by sorting edges).

Traps. (1) In Prim's you only ever consider edges with **exactly one** endpoint in the tree — an edge fully inside the tree would make a cycle. (2) Update the "cheapest edge to each outside vertex" after every addition. (3) Don't mix the two algorithms' bookkeeping.

---

6. Comparison Counting Sort

The idea. A *Space–Time trade-off*: instead of moving elements around by comparison, **count** for each element how many others are smaller than it. That count **is** the element's final index. One extra array of counts buys you a direct placement.

Analogy. Assigning race finishers to podium positions by tallying wins. For each runner you ask "how many runners did you beat?" If you beat 3 others, you finish in position 3 (0-indexed). Every runner's beat-count is their finishing slot — no need to line them up and compare again.

Algorithm. For each pair (i, j) with i < j: if A[i] < A[j] increment Count[j], else increment Count[i]. After all pairs, Count[i] = number of elements smaller than A[i]. Place each A[i] into output position Count[i].

Worked example — `[31, 10, 24, 1, 12, 5]`.

Final counts (elements smaller than each):

Element	31	10	24	1	12	5
Count (# smaller)	5	2	4	0	3	1

Placing each element at its Count index → **Sorted: `[1, 5, 10, 12, 24, 31]`.**

Traps. (1) Count[i] = number **strictly smaller**; with distinct values it's unambiguous, but duplicates need a consistent tie rule (the pairwise "else increment Count[i]" handles it deterministically). (2) It's Θ(n²) because of the nested pairwise comparison — **do not** confuse it with **distribution** counting sort, which is Θ(n + range) and works by tallying value frequencies (only for small integer ranges). The deck's question is the **comparison** variant.

---

7. Coin-Row Problem — Dynamic Programming

The idea. A row of n coins with values $c_1 \ldots c_n$. Pick coins for **maximum total value**, but **no two adjacent** coins. This is the archetypal DP: define a named quantity for a sub-instance, find a recurrence, fill a table bottom-up. Your coin-row derivation nails the reasoning.

Analogy. Walking down a buffet where you may take dishes, but never two dishes sitting **next to each other**. At each dish you face one decision: **take it** (and you're barred from the dish right before it) or **skip it** (and keep whatever you'd accumulated up to the previous dish). You want the richest plate overall.

The recurrence. Let F(i) = the max amount obtainable from the first i coins. - **Take coin i:** you get c_i plus the best from coins 1…i−2 (coin i−1 is forbidden) → c_i + F(i−2). - **Skip coin i:** you keep F(i−1). - **$F(i) = \max(F(i-1), c_i + F(i-2))$**, with F(0) = 0, F(1) = c_1.

> Why F(i−2) and not F(i−1) when taking coin i? Because taking coin i **bans its neighbor** i−1. Jumping to i−2 guarantees you skip the forbidden neighbor — this is the whole "no two adjacent" rule encoded in one index.

Worked example — coins `{3, 2, 5, 2, 1, 2, 4}`.

i	0	1	2	3	4	5	6	7
c_i	—	3	2	5	2	1	2	4
F(i)	0	3	3	8	8	9	10	**13**

Reading each cell: F(3)=max(F(2)=3, 5+F(1)=8)=8; F(5)=max(F(4)=8, 1+F(3)=9)=9; F(7)=max(F(6)=10, 4+F(5)=13)=**13**.

Maximum = 13. Recover the selection by walking back: whenever F(i) ≠ F(i−1), coin i was taken → **coins at positions 1, 3, 5, 7 = values 3, 5, 1, 4** (sum 13). *(Check the exercise `{7,3,5,12,2,8}` yourself: table 0,7,7,12,19,19,27 → max 27, coins at positions 1, 4, 6 = 7+12+8.)*

Traps. (1) The base cases F(0)=0 and F(1)=c_1 anchor everything — get them wrong and the whole row shifts. (2) To **recover** which coins were chosen, compare F(i) with F(i−1): equal ⇒ coin i skipped; greater ⇒ coin i taken, then jump back **two**. (3) This is DP, not greedy — grabbing the largest coins greedily can violate adjacency and lose.

---

8. Money Change — Minimum Coins (DP)

The idea. Given coin denominations and a target amount, use the **fewest coins** to make exactly that amount (unlimited supply of each). Another bottom-up DP, this time indexed by **amount** rather than position.

Analogy. Filling a jug to an exact line using cups of fixed sizes, minimizing pours. To reach the 10-mark, you ask: for each cup size c, "what's the fewest pours to reach 10 − c?" — then add one pour for cup c. The best over all cups wins.

The recurrence. Let F(m) = fewest coins to make amount m. - **$F(m) = \min$ over each coin $c_j \le m$ of $(F(m - c_j) + 1)$**, with F(0) = 0.

Worked example — coins `[1, 2, 5]`, amount 10.

m	0	1	2	3	4	5	6	7	8	9	10
F(m)	0	1	1	2	2	1	2	2	3	3	**2**

E.g. F(5)=min(F(4)+1, F(3)+1, F(0)+1)=1 (one nickel); F(10)=min(F(9)+1, F(8)+1, F(5)+1)= F(5)+1 = **2**.

Minimum = 2 coins (two 5s). To recover coins: at each m, pick the coin c_j that achieved the min and step back to m − c_j.

Traps. (1) Initialize F(0)=0 and all others to **∞** so unreachable amounts stay unreachable. (2) This DP is guaranteed optimal for **any** denomination set — unlike the **greedy** change-making in your greedy note, which only works for "nice" coin systems and can fail otherwise (e.g. coins {1,3,4}, amount 6: greedy gives 4+1+1=3 coins, DP gives 3+3=2). (3) Complexity Θ(n · amount) — pseudo-polynomial, because it scales with the **value** of the amount, not its input size.

---

9. Assignment Problem — Branch-and-Bound

The idea. Assign n people to n jobs (one each) to **minimize total cost**, given a cost matrix C where C[i][j] is the cost of person i doing job j. Brute force is n! permutations. **Branch-and-bound** builds a **state-space tree** of partial assignments and **prunes** any branch whose optimistic lower bound already exceeds the best complete solution found — so it explores far fewer than n! nodes.

Analogy. Planning a wedding seating chart by trying arrangements, but with a running "best so far" and a smart shortcut: before fully seating a table, you compute the **cheapest it could possibly get** from here. If even that optimistic estimate is worse than an arrangement you've already completed, you abandon the whole branch unseen. You never waste time on a table that can't win.

Algorithm. 1. **Lower bound (root & each node):** for a partial assignment, add the costs already committed, plus for each **unassigned person** the **smallest** remaining entry in their row. This is optimistic — it assumes everyone else gets their personal best — so it never overestimates. 2. **Branch:** pick the next person; create a child node for each job still available. 3. **Bound & prune:** compute each child's lower bound; explore the most promising (smallest bound) first (best-first). Prune any node whose bound ≥ current best complete solution. 4. When a **leaf** (full assignment) beats the incumbent, update the best. Continue until every live node is pruned or explored.

Worked example — 4×4 cost matrix (rows = persons a,b,c,d; cols = jobs 1–4):

	Job 1	Job 2	Job 3	Job 4
a	9	2	7	8
b	6	4	3	7
c	5	8	1	8
d	7	6	9	4

Root lower bound = sum of each row's minimum = 2 + 3 + 1 + 4 = **10** (an optimistic floor, not yet feasible — it double-uses jobs). Branch on person **a**'s four jobs, bound each, expand the cheapest, and prune. The search converges to the optimal assignment:

a → Job 2 (2), b → Job 1 (6), c → Job 3 (1), d → Job 4 (4) — total cost = 13.

Every other complete assignment costs ≥ 13; branch-and-bound reaches 13 while pruning most of the 24 permutations.

Traps. (1) The lower bound must be **optimistic** (≤ true cost) or you'd prune the real answer — using row minima is the standard bound. (2) It's for **minimization** here; for maximization you'd use an upper bound instead. (3) The state-space **tree drawing** with each node's bound is what the exam grades — show the bound at every node and mark pruned branches. (4) Branch-and-bound still has exponential worst case; pruning just makes it fast in practice.

Final-Week Drill Plan

Work in this order — hardest-to-recall last so it's freshest:

1. **Redo every worked example above from a blank page.** If a number doesn't match, reread that section's "Traps."
2. **Do the deck's exercise problems** (Hoare on `[55,12,78,64,3,39,22,81]`; heap on `[1,3,5,4,6,13,10,9,8,15,17]`; coin-row `{7,3,5,12,2,8}`; the second assignment matrix) — solutions/patterns are all derivable from the sections here.
3. **Memorize the complexity table** and the technique map — these are free points and frame every question.
4. **Rehearse the four "vs" distinctions:** DP vs D&C, Greedy vs DP, Prim vs Kruskal, comparison vs distribution counting sort.

> Linked notes to revisit: Final Exam · Greedy Technique · Coin Change Problem · Cyclic vs Acyclic Graph · DFS_PR~1